



Numa

Calculadora criptográfica

Ejemplos resueltos

Un ejemplo explicado por cada uno de los 26 submenús de la calculadora: Enunciado, datos, idea del método, procedimiento paso a paso, tabla y comprobación. Todos los resultados se calculan con el mismo código que usa la app.

Realizado por Leydy Yohana Macareo Fuentes

UNAB, clase de Ciberseguridad

Índice

Cada ejemplo tiene seis partes: Enunciado, Datos, Idea del método, Procedimiento, Tabla y Resultado con su comprobación. Los resultados no están escritos a mano: se calculan con el mismo código de la calculadora.

Módulo 1: Matemática modular	3
1.1 Módulo de un número negativo: $-7 \bmod 26$	3
1.2 Inverso aditivo de 7 módulo 26	4
1.3 Recuperar A a partir de $C = 1100$ y $K = 0110$	4
1.4 $\text{MCD}(7, 26)$ con el algoritmo de Euclides	6
1.5 Inverso de 3 módulo 7 probando uno a uno	7
1.6 Inverso de 7 módulo 26 con la tabla del AEE	8
Módulo 2: Criptografía clásica	10
2.1 Cifrar HOLA con $k = 3$ en módulo 27	10
2.2 César de HOLA con $k = 3$ y alfabeto de 26 letras	11
2.3 Vernam de HOLA con la clave XMCK	12
2.4 Atbash de HOLA: Reflejar el alfabeto	14
2.5 Transposición columnar de HOLAMUNDO con la clave CLAVE	15
2.6 Afín de HOLA con $a = 5$ y $b = 8$	16
2.7 Sustitución simple de HOLA con la palabra MURCIELAGO	18
Módulo 3: Criptografía moderna	20
3.1 Diffie-Hellman con $p = 23$, $g = 5$, $a = 6$ y $b = 15$	20
3.2 RSA con $p = 61$, $q = 53$, $e = 17$ y $m = 65$	22
3.3 $3^{13} \bmod 7$ con cuadrados sucesivos	24
Módulo 4: Algoritmos hash	26
4.1 MD5 de «abc» y efecto avalancha frente a «abd»	26
4.2 SHA-256 de «abc»	27
4.3 SHA-512 de «abc»	28
Módulo 5: Codificación	30
5.1 «Hola» en códigos ASCII decimales	30
5.2 «Hola» en hexadecimal	31
5.3 «Hola» en binario de 8 bits por byte	32
5.4 «Hola» en Base64	33
Módulo 6: Uso de salt	35
6.1 Verificar «Clave123» con el salt a1b2c3d4 y MD5	35
6.2 Verificar «Clave123» con el salt a1b2c3d4 y SHA-256	36
6.3 Verificar «Clave123» con el salt a1b2c3d4 y SHA-512	37

1 Matemática modular

La aritmética modular trabaja con residuos: Dos números son «iguales módulo n » cuando dejan el mismo residuo al dividirlos entre n . De aquí salen el inverso aditivo, el inverso multiplicativo, el máximo común divisor y el algoritmo extendido de Euclides, las herramientas que después usan los cifrados clásicos y RSA.

1.1 Módulo de un número negativo: $-7 \bmod 26$

Enunciado

Calcular $-7 \bmod 26$, es decir, el residuo de dividir -7 entre 26 , y escribir la identidad $a = q \times n + b$ con $0 \leq b < n$.

Datos

Número a	-7
Módulo n	26

Idea del método

« $a \bmod n$ » es el residuo que queda al dividir a entre n . Siempre está entre 0 y $n - 1$, también cuando a es negativo: Por eso el cociente se toma hacia abajo (-1 en vez de 0). La identidad $a = q \times n + b$ conecta el cociente q , el módulo n y el residuo b .

$$a = q \times n + b, \text{ con } 0 \leq b < n$$

Procedimiento

- 1. Qué buscamos.** El residuo de dividir -7 entre 26 : Lo que sobra, un número b que siempre queda entre 0 y 25 . Eso es lo que significa « $-7 \bmod 26$ ».
- 2. Dividimos.** -7 entre 26 cabe -1 veces (ese es el cociente q). Como a es negativo, el cociente se toma hacia abajo para que el residuo no salga negativo. Entonces $q \times n = -1 \times 26 = -26$.
- 3. Calculamos el residuo.** $b = a - q \times n = -7 - (-26) = 19$.
- 4. Comprobación.** $-1 \times 26 + 19 = -7$, y 19 está entre 0 y 25 , como debe ser.

Resultado

19

Comprobación: $-7 = -1 \times 26 + 19$, con $0 \leq 19 < 26$

1.2 Inverso aditivo de 7 módulo 26

Enunciado

Hallar el inverso aditivo de 7 en módulo 26 y comprobar que la suma es un múltiplo de 26.

Datos

Número a	7
Módulo n	26

Idea del método

El inverso aditivo de a es el número x que, sumado a a, da 0 en módulo n: $a + x = 0 \pmod{n}$. Se calcula como $-a \bmod n$, que para $0 < a < n$ equivale a $n - a$. Es la versión modular del opuesto de un número.

$$x = -a \bmod n, \text{ de modo que } a + x = 0 \pmod{n}$$

Procedimiento

- 1. Qué buscamos.** Un número x entre 0 y 25 que sumado a 7 dé un múltiplo de 26, es decir $7 + x = 0 \pmod{26}$. Ese x es el inverso aditivo.
- 2. Calculamos.** $x = -a \bmod n = -7 \bmod 26$. Como 7 está entre 1 y 25, basta restarlo de 26: $26 - 7 = 19$.
- 3. Comprobación.** $7 + 19 = 26$, y $26 \bmod 26 = 0$: La suma es un múltiplo de 26.

Resultado

19

Comprobación: $7 + 19 = 26 = 0 \pmod{26}$

1.3 Recuperar A a partir de C = 1100 y K = 0110

Enunciado

Dados el mensaje cifrado C = 1100 y la clave K = 0110, en binario, recuperar el mensaje original A.

Datos

Mensaje cifrado C	1100
Clave K	0110
Base de las entradas	Binario

Idea del método

XOR (XOR) compara dos bits y da 1 cuando son distintos y 0 cuando son iguales. Su propiedad clave es que se deshace a sí mismo: $(A \text{ XOR } K) \text{ XOR } K = A$. Por eso, si el cifrado fue $C = A \text{ XOR } K$, basta aplicar de nuevo la clave para recuperar A. Es la base del cifrado Vernam (2.3).

$$A = C \text{ XOR } K$$

Procedimiento

1. **Qué buscamos.** El mensaje original A. El cifrado fue $C = A \text{ XOR } K$ (XOR con la clave K), y XOR se deshace aplicándolo otra vez con la misma clave: $A = C \text{ XOR } K$.
2. **Pasamos todo a binario.** Con 4 bits: $C = 1100$ y $K = 0110$.
3. **Aplicamos XOR bit a bit.** Comparamos cada bit de C con el bit de K que está en la misma posición: Si son distintos el resultado es 1, si son iguales es 0. El primer bit: 1 frente a 0 da 1.
4. **Resultado.** $A = C \text{ XOR } K = 1010$.
5. **Comprobación.** $A \text{ XOR } K = 1010 \text{ XOR } 0110 = 1100 = C$: Aplicar XOR otra vez devuelve el mensaje cifrado.

Detalle

$$\begin{array}{r} C \quad 1100 \\ \text{XOR } K \quad 0110 \\ \hline A \quad 1010 \end{array}$$

Tabla

Valor	Binario	Decimal	Hexadecimal
C	1100	12	C
K	0110	6	6
$A = C \text{ XOR } K$	1010	10	A

Cada fila muestra el mismo valor en las tres bases; el XOR se hace sobre la columna de binario.

Resultado

1010

Comprobación: $A \text{ XOR } K = C$: $1010 \text{ XOR } 0110 = 1100$, se recupera C

1.4 MCD(7, 26) con el algoritmo de Euclides

Enunciado

Calcular MCD(7, 26) con el algoritmo de Euclides y decidir si 7 tiene inverso multiplicativo módulo 26.

Datos

Número a	7
Número b	26

Idea del método

El algoritmo de Euclides encuentra el máximo común divisor dividiendo una y otra vez: El divisor de cada paso pasa a ser el dividendo del siguiente y el residuo, el nuevo divisor, hasta que el residuo es 0. El último residuo distinto de cero es el MCD. Dos números con MCD 1 se llaman coprimos, y un número tiene inverso módulo n exactamente cuando es coprimo con n.

$$a = q \times b + r, \text{ repetido hasta que } r = 0$$

Procedimiento

- 1. Qué buscamos.** El máximo común divisor de 7 y 26: El número más grande que divide a los dos sin dejar residuo. Euclides lo encuentra con divisiones sucesivas.
- 2. Ordenamos.** Como $7 < 26$, empezamos dividiendo 26 entre 7.
- 3. División 1.** 26 entre 7 cabe 3 veces y sobra 5: $26 = 3 \times 7 + 5$. Ahora dividimos 7 entre 5.
- 4. División 2.** 7 entre 5 cabe 1 vez y sobra 2: $7 = 1 \times 5 + 2$. Ahora dividimos 5 entre 2.
- 5. División 3.** 5 entre 2 cabe 2 veces y sobra 1: $5 = 2 \times 2 + 1$. Ahora dividimos 2 entre 1.
- 6. División 4.** 2 entre 1 cabe 2 veces y sobra 0: $2 = 2 \times 1 + 0$. El residuo es 0: Aquí paramos.
- 7. Resultado.** El último residuo distinto de cero es 1, así que $\text{MCD}(7, 26) = 1$.
- 8. Qué significa.** Como el MCD es 1, 7 y 26 no comparten divisores (son coprimos) y 7 sí tiene inverso módulo 26.
- 9. Comprobación.** $7 = 7 \times 1$ y $26 = 26 \times 1$: 1 divide exacto a los dos.

Tabla

Paso	a	b	Cociente q	Residuo r
1	26	7	3	5
2	7	5	1	2
3	5	2	2	1
4	2	1	2	0

En cada fila se divide a entre b y el residuo pasa a ser el b de la fila siguiente. El MCD es el último residuo distinto de cero.

Resultado

1

Comprobación: 1 divide a los dos: $7 = 7 \times 1$ y $26 = 26 \times 1$

1.5 Inverso de 3 módulo 7 probando uno a uno

Enunciado

Hallar el inverso multiplicativo de 3 en módulo 7 probando $x = 1, 2, 3, \dots$ hasta que $3 \times x$ deje residuo 1.

Datos

Número a	3
Módulo n	7

Idea del método

El inverso multiplicativo de a módulo n es el x que cumple $a \times x = 1 \pmod{n}$. El método tradicional lo busca por fuerza bruta: Prueba cada x desde 1 hasta dar con el residuo 1. Sirve con módulos pequeños; para módulos grandes se usa el algoritmo extendido de Euclides (1.6).

$$a \times x = 1 \pmod{n}$$

Procedimiento

- 1. Qué buscamos.** Un número x entre 1 y 6 tal que $3 \times x$ deje residuo 1 al dividirlo entre 7. Ese x es el inverso de 3 módulo 7.
- 2. Cómo lo hacemos.** A fuerza bruta: Probamos $x = 1, 2, 3, \dots$ y en cada intento calculamos el producto y su residuo, hasta dar con residuo 1.
- 3. Probamos $x = 1$.** $3 \times 1 = 3$, y $3 \bmod 7 = 3$. No es 1, seguimos.
- 4. Probamos $x = 2$.** $3 \times 2 = 6$, y $6 \bmod 7 = 6$. No es 1, seguimos.
- 5. Probamos $x = 3$.** $3 \times 3 = 9$, y $9 \bmod 7 = 2$. No es 1, seguimos.
- 6. Probamos $x = 4$.** $3 \times 4 = 12$, y $12 \bmod 7 = 5$. No es 1, seguimos.
- 7. Probamos $x = 5$.** $3 \times 5 = 15$, y $15 \bmod 7 = 1$. El residuo es 1: Encontrado.
- 8. Resultado.** El inverso de 3 módulo 7 es 5; hicieron falta 5 intentos.
- 9. Comprobación.** $3 \times 5 = 15 = 2 \times 7 + 1$: El residuo es 1.

Tabla

Intento x	$a \times x$	$a \times x \bmod n$
1	3	3
2	6	6
3	9	2
4	12	5
5	15	1

$x = 5$ es el primer intento cuyo residuo es 1: Ese es el inverso.

Resultado

5

Comprobación: $3 \times 5 = 15 = 1 \pmod{7}$

1.6 Inverso de 7 módulo 26 con la tabla del AEE

Enunciado

Hallar el inverso multiplicativo de 7 en módulo 26 con la tabla del algoritmo extendido de Euclides (AEE) y comprobar el resultado.

Datos

Número a	7
Módulo n	26

Idea del método

El AEE hace las mismas divisiones de Euclides, pero además lleva dos columnas s y t que en cada fila cumplen $s \times n + t \times a = r$. Cuando el residuo r llega a 1, la identidad dice $s \times n + t \times a = 1$, y como $s \times n$ es múltiplo de n, queda $t \times a = 1 \pmod{n}$; t es el inverso de a. Si t sale negativo se le suma n. RSA (3.2) y el cifrado afín (2.6) usan esta misma tabla.

$$s \times n + t \times a = 1 \Rightarrow a^{-1} = t \bmod n$$

Procedimiento

1. **Qué buscamos.** Un número x entre 1 y 25 tal que $7 \times x$ deje residuo 1 al dividirlo entre 26. Ese x es el inverso de 7 módulo 26.

2. **Preparamos la tabla.** Dos filas de inicio: La de 26 ($s = 1, t = 0$) y la de 7 ($s = 0, t = 1$). En cada fila se cumple $s \times 26 + t \times 7 = \text{residuo}$; s y t llevan la cuenta de cómo se forma cada residuo a partir de los dos números.
3. **Ronda 1.** Dividimos 26 entre 7: Cabe 3 veces y sobra 5, así que $26 = 3 \times 7 + 5$. Los nuevos s y t salen de restar 3 veces la fila anterior: $s = 1 - 3 \times 0 = 1$ y $t = 0 - 3 \times 1 = -3$.
4. **Ronda 2.** Dividimos 7 entre 5: Cabe 1 vez y sobra 2, así que $7 = 1 \times 5 + 2$. Los nuevos s y t salen de restar 1 vez la fila anterior: $s = 0 - 1 \times 1 = -1$ y $t = 1 - 1 \times (-3) = 4$.
5. **Ronda 3.** Dividimos 5 entre 2: Cabe 2 veces y sobra 1, así que $5 = 2 \times 2 + 1$. Los nuevos s y t salen de restar 2 veces la fila anterior: $s = 1 - 2 \times (-1) = 3$ y $t = (-3) - 2 \times 4 = -11$. Como el residuo es 1, esta es la fila que buscábamos.
6. **Ronda 4.** Dividimos 2 entre 1: Cabe 2 veces y sobra 0, así que $2 = 2 \times 1 + 0$. Los nuevos s y t salen de restar 2 veces la fila anterior: $s = (-1) - 2 \times 3 = -7$ y $t = 4 - 2 \times (-11) = 26$. Al llegar a residuo 0 se acaba la tabla: 4 divisiones, 4 rondas.
7. **Leemos el inverso.** En la fila con residuo 1, $t = -11$. Eso significa que $3 \times 26 + (-11) \times 7 = 1$, o sea que -11×7 deja residuo 1 al dividirlo entre 26.
8. **Lo pasamos a positivo.** -11 no está entre 0 y 25, así que le sumamos 26: $-11 + 26 = 15$. Sumar múltiplos de 26 no cambia el residuo.
9. **Resultado.** El inverso de 7 módulo 26 es 15.
10. **Comprobación.** $7 \times 15 = 105$ y $105 = 4 \times 26 + 1$: El residuo es 1.

Tabla

Ronda	Cociente q	Residuo r	s	t
Inicio	—	26	1	0
Inicio	—	7	0	1
1	3	5	1	-3
2	1	2	-1	4
3	2	1	3	-11
4	2	0	-7	26

El inverso es el t de la fila con residuo $r = 1$: $t = -11$, y $-11 \bmod 26 = 15$.

Resultado

15

Comprobación: $7 \times 15 = 105 = 1 \pmod{26}$

2 Criptografía clásica

Los cifrados clásicos transforman letras: Las desplazan (módulo 27 y César), las reflejan (Atbash), las multiplican (afín), las sustituyen (sustitución simple) o cambian su orden (transposición). Vernam trabaja con bits. Todos usan el alfabeto español de 27 letras salvo que se indique lo contrario, y todos se deshacen con la operación inversa.

2.1 Cifrar HOLA con $k = 3$ en módulo 27

Enunciado

Cifrar el mensaje HOLA con desplazamiento $k = 3$ en el alfabeto español de 27 letras.

Datos

Mensaje	HOLA
Desplazamiento k	3
Modo	Cifrar

Idea del método

El cifrado en módulo 27 convierte cada letra en su posición ($A = 0 \dots \tilde{N} = 14 \dots Z = 26$), le suma el desplazamiento k y toma el residuo módulo 27 para no salirse del alfabeto. Descifrar es restar k . Es el cifrado de César adaptado al alfabeto con $\tilde{N}$.

$$C = (M + k) \bmod 27$$

$$M = (C - k) \bmod 27$$

Procedimiento

- 1. Qué hacemos.** Cada letra se cambia por la que está 3 puestos más adelante en el alfabeto de 27 letras. Si nos pasamos de la Z seguimos por la A: Eso es «mod 27».
- 2. Letras a números.** $A = 0, B = 1 \dots \tilde{N} = 14 \dots Z = 26$. «HOLA» queda $H = 7, O = 15, L = 11, A = 0$.
- 3. Sumamos 3 y tomamos mod 27.** $H: (7 + 3) \bmod 27 = 10 \rightarrow K$. $O: (15 + 3) \bmod 27 = 18 \rightarrow R$. $L: (11 + 3) \bmod 27 = 14 \rightarrow \tilde{N}$. $A: (0 + 3) \bmod 27 = 3 \rightarrow D$.
- 4. Resultado.** «HOLA» se convierte en «KRÑD».
- 5. Comprobación.** Para deshacerlo se resta 3 a cada número: $K = 10 \rightarrow (10 - 3) \bmod 27 = 7 \rightarrow H$, y así se recupera «HOLA».

Tabla

Letra	Número	Operación	Letra cifrada
H	7	$(7 + 3) \bmod 27 = 10$	K

Letra	Número	Operación	Letra cifrada
O	15	$(15 + 3) \bmod 27 = 18$	R
L	11	$(11 + 3) \bmod 27 = 14$	Ñ
A	0	$(0 + 3) \bmod 27 = 3$	D

Cada fila convierte la letra en número, aplica la fórmula y vuelve a letra.

Resultado

KRÑD

Comprobación: $(10 - 3) \bmod 27 = 7 = H$: La operación inversa recupera la primera letra

2.2 César de HOLA con $k = 3$ y alfabeto de 26 letras

Enunciado

Cifrar HOLA con el cifrado de César, $k = 3$, usando el alfabeto inglés de 26 letras (sin Ñ).

Datos

Mensaje	HOLA
Desplazamiento k	3
Alfabeto	26 letras
Modo	Cifrar

Idea del método

El cifrado de César desplaza cada letra k posiciones. Con 26 letras el módulo es 26 y la Ñ no existe (se escribe como N). Solo hay 25 desplazamientos posibles, así que se rompe probándolos todos: Por eso hoy es solo didáctico.

$$C = (M + k) \bmod 26$$

$$M = (C - k) \bmod 26$$

Procedimiento

- 1. Qué hacemos.** Cada letra se cambia por la que está 3 puestos más adelante en el alfabeto de 26 letras. Si nos pasamos de la Z seguimos por la A: Eso es «mod 26».
- 2. Letras a números.** $A = 0, B = 1 \dots Z = 25$. «HOLA» queda $H = 7, O = 14, L = 11, A = 0$.
- 3. Sumamos 3 y tomamos mod 26.** $H: (7 + 3) \bmod 26 = 10 \rightarrow K$. $O: (14 + 3) \bmod 26 = 17 \rightarrow R$. $L: (11 + 3) \bmod 26 = 14 \rightarrow O$. $A: (0 + 3) \bmod 26 = 3 \rightarrow D$.

4. **Resultado.** «HOLA» se convierte en «KROD».
5. **Comprobación.** Para deshacerlo se resta 3 a cada número: $K = 10 \rightarrow (10 - 3) \bmod 26 = 7 \rightarrow H$, y así se recupera «HOLA».

Tabla

Letra	Número	Operación	Letra cifrada
H	7	$(7 + 3) \bmod 26 = 10$	K
O	14	$(14 + 3) \bmod 26 = 17$	R
L	11	$(11 + 3) \bmod 26 = 14$	O
A	0	$(0 + 3) \bmod 26 = 3$	D

Cada fila convierte la letra en número, aplica la fórmula y vuelve a letra.

Resultado

KROD

Comprobación: $(10 - 3) \bmod 26 = 7 = H$: La operación inversa recupera la primera letra

2.3 Vernam de HOLA con la clave XMCK

Enunciado

Cifrar HOLA con Vernam usando la clave XMCK y mostrar el resultado en hexadecimal y en binario.

Datos

Mensaje	HOLA
Clave	XMCK
Modo	Cifrar

Idea del método

Vernam aplica XOR entre cada byte del mensaje y el byte de la clave que está en la misma posición; la clave debe medir lo mismo que el mensaje. Como XOR se deshace a sí mismo, descifrar es repetir la misma operación con la misma clave. Con una clave aleatoria, secreta y de un solo uso es el «cuaderno de un solo uso», el único cifrado matemáticamente irrompible.

$$C = M \text{ XOR } K$$

$$M = C \text{ XOR } K$$

Procedimiento

- 1. Qué hacemos.** Vernam mezcla el mensaje con una clave del mismo tamaño usando XOR bit a bit. Cada carácter se convierte en un byte (8 bits, en UTF-8) y se combina con el byte de la clave que está en la misma posición.
- 2. El mensaje en bytes.** En UTF-8, «HOLA» ocupa 4 bytes (en hexadecimal): «H» = 48, «O» = 4F, «L» = 4C, «A» = 41.
- 3. La clave en bytes.** «XMCK» ocupa 4 bytes, igual que el mensaje: «X» = 58, «M» = 4D, «C» = 43, «K» = 4B.
- 4. XOR byte a byte.** Comparamos bit a bit cada byte del mensaje con el byte de la clave que está en la misma posición: 1 si los bits son distintos, 0 si son iguales. El primer byte: 48 XOR 58 = 01001000 XOR 01011000 = 00010000, que en hexadecimal es 10. La tabla muestra los 4 bytes.
- 5. Resultado.** Juntamos los bytes en hexadecimal: 10020F0A. En binario: 00010000 00000010 00001111 00001010.
- 6. Comprobación.** Aplicar XOR con la misma clave deshace el cifrado: 10 XOR 58 = 48, el primer byte del mensaje.

Detalle

Hexadecimal: 10020F0A

Binario: 00010000 00000010 00001111 00001010

Tabla

Byte	Mensaje (bin)	Clave (bin)	Resultado (bin)	Resultado (hex)
1	01001000	01011000	00010000	10
2	01001111	01001101	00000010	02
3	01001100	01000011	00001111	0F
4	01000001	01001011	00001010	0A

Cada fila muestra un byte del mensaje, el byte de la clave en la misma posición y su XOR: 1 si los bits son distintos, 0 si son iguales.

Resultado

10020F0A

Comprobación: 10 XOR 58 = 48: Aplicar XOR con la clave otra vez recupera el primer byte

2.4 Atbash de HOLA: Reflejar el alfabeto

Enunciado

Cifrar HOLA con Atbash, el cifrado que refleja el alfabeto.

Datos

Mensaje	HOLA
Modo	Cifrar

Idea del método

Atbash cambia la primera letra por la última, la segunda por la penúltima y así sucesivamente: La letra de la posición i pasa a la $26 - i$. Reflejar dos veces devuelve el original, así que cifrar y descifrar son la misma operación. En el alfabeto de 27 letras la N (posición 13) es el centro y no cambia.

$$C = 26 - M$$

Procedimiento

- 1. Qué hacemos.** Atbash refleja el alfabeto: La primera letra se cambia por la última, la segunda por la penúltima, y así. Con los números del alfabeto de 27, la letra i se cambia por $26 - i$. Cifrar y descifrar son la misma operación.
- 2. Letras a números.** A = 0, B = 1 ... Ñ = 14 ... Z = 26. «HOLA» queda H = 7, O = 15, L = 11, A = 0.
- 3. Reflejamos cada letra.** H: $26 - 7 = 19 \rightarrow$ S. O: $26 - 15 = 11 \rightarrow$ L. L: $26 - 11 = 15 \rightarrow$ O. A: $26 - 0 = 26 \rightarrow$ Z.
- 4. La N no cambia.** Está en la posición 13, justo en el centro del alfabeto: $26 - 13 = 13$.
- 5. Resultado.** «HOLA» se convierte en «SLOZ».
- 6. Comprobación.** Reflejar otra vez devuelve el original: S = 19 $\rightarrow$ $26 - 19 = 7 \rightarrow$ H, y así se recupera «HOLA».

Tabla

Letra	Número	Operación	Letra cifrada
H	7	$26 - 7 = 19$	S
O	15	$26 - 15 = 11$	L
L	11	$26 - 11 = 15$	O
A	0	$26 - 0 = 26$	Z

Atbash refleja el alfabeto: A $\leftrightarrow$ Z, B $\leftrightarrow$ Y... y la N, en el centro, se queda en su sitio.

Resultado

SLOZ

Comprobación: $26 - 19 = 7 = H$: Reflejar otra vez recupera la primera letra

2.5 Transposición columnar de HOLAMUNDO con la clave CLAVE

Enunciado

Cifrar HOLAMUNDO por transposición columnar simple con la clave CLAVE.

Datos

Mensaje	HOLAMUNDO
Clave (palabra)	CLAVE
Modo	Cifrar

Idea del método

La transposición no cambia las letras, cambia su orden. El mensaje se escribe en filas bajo la clave, se rellena con X hasta completar la última fila y se leen las columnas en el orden alfabético de las letras de la clave. Para descifrar se reparte el texto en columnas con ese mismo orden y se lee por filas.

las columnas se leen en el orden alfabético de la clave

Procedimiento

- 1. Qué hacemos.** Escribimos el mensaje en filas debajo de la clave «CLAVE», una letra por columna, y luego leemos las columnas en el orden alfabético de las letras de la clave.
- 2. Numeramos las columnas.** Por orden alfabético de las letras de la clave: A=1, C=2, E=3, L=4, V=5. Si una letra se repite, va primero la de más a la izquierda.
- 3. Llenamos la matriz.** El mensaje tiene 9 letras y la clave 5, así que salen 2 filas de 5 letras; falta 1 casilla y la rellenamos con X: «HOLAMUNDOX». Filas: HOLAM / UNDOX.
- 4. Leemos las columnas en orden.** Columna por columna, de arriba abajo: A(1) -> LD, C(2) -> HU, E(3) -> MX, L(4) -> ON, V(5) -> AO.
- 5. Resultado.** Uniendo los trozos en ese orden queda «LDHUMXONAO».
- 6. Comprobación.** La matriz conserva las 10 letras: Leída por filas devuelve «HOLAMUNDOX», el mensaje con su relleno.

Tabla

C (2)	L (4)	A (1)	V (5)	E (3)
H	O	L	A	M
U	N	D	O	X

La matriz con la clave y su número de orden encima: Al cifrar se leen las columnas siguiendo esa numeración; al descifrar se leen las filas.

Resultado

LDHUMXONAO

Comprobación: La matriz conserva las 10 letras: Leerla por filas devuelve «HOLAMUNDOX»

2.6 Afín de HOLA con $a = 5$ y $b = 8$

Enunciado

Cifrar HOLA con el cifrado afín de llaves $a = 5$ y $b = 8$ en el alfabeto de 27 letras.

Datos

Mensaje	HOLA
Llave a	5
Llave b	8
Modo	Cifrar

Idea del método

El afín combina una multiplicación y una suma: $C = (a \times M + b) \bmod 27$. Para descifrar hay que deshacer la suma y luego la multiplicación, y eso exige el inverso a^{-1} de a módulo 27, que existe solo si $\text{MCD}(a, 27) = 1$. El inverso se obtiene con la tabla del AEE (1.6).

$$C = (a \times M + b) \bmod 27$$

$$M = a^{-1} \times (C - b) \bmod 27$$

Procedimiento

1. **Qué hacemos.** El afín multiplica y suma: $C = (a \times M + b) \bmod 27$, con $a = 5$ y $b = 8$. Para poder descifrar después hace falta a^{-1} , el inverso de a módulo 27, y por eso a debe ser coprimo con 27.

2. **Comprobamos a.** $\text{MCD}(5, 27) = 1$: a no comparte divisores con 27, así que tiene inverso.
3. **Hallamos a^{-1} .** Con la tabla del Euclides extendido del submenú 1.6 (3 rondas): $t = 11$ y $a^{-1} = 11 \bmod 27 = 11$. Comprobación: $5 \times 11 = 55$ y $55 \bmod 27 = 1$.
4. **Letras a números.** A = 0, B = 1 ... Ñ = 14 ... Z = 26. «HOLA» queda H = 7, O = 15, L = 11, A = 0.
5. **Aplicamos $C = (a \times M + b) \bmod 27$.** H: $(5 \times 7 + 8) \bmod 27 = 16 \rightarrow$ P. O: $(5 \times 15 + 8) \bmod 27 = 2 \rightarrow$ C. L: $(5 \times 11 + 8) \bmod 27 = 9 \rightarrow$ J. A: $(5 \times 0 + 8) \bmod 27 = 8 \rightarrow$ I.
6. **Resultado.** «HOLA» se convierte en «PCJI».
7. **Comprobación.** Para descifrar: $P = 16 \rightarrow 11 \times (16 - 8) \bmod 27 = 7 \rightarrow$ H, y así se recupera «HOLA».

Tabla

Letra	Número	Operación	Letra cifrada
H	7	$(5 \times 7 + 8) \bmod 27 = 16$	P
O	15	$(5 \times 15 + 8) \bmod 27 = 2$	C
L	11	$(5 \times 11 + 8) \bmod 27 = 9$	J
A	0	$(5 \times 0 + 8) \bmod 27 = 8$	I

Cada fila convierte la letra en número, aplica la fórmula afín y vuelve a letra.

Tabla del AEE para $a^{-1} = 11$

Ronda	Cociente q	Residuo r	s	t
Inicio	—	27	1	0
Inicio	—	5	0	1
1	5	2	1	-5
2	2	1	-2	11
3	2	0	5	-27

La misma tabla del submenú 1.6 con $n = 27$ y $a = 5$: El inverso es el t de la fila con residuo $r = 1$, y $11 \bmod 27 = 11$.

Resultado

PCJI

Comprobación: $11 \times (16 - 8) \bmod 27 = 7 = \text{H}$: La operación inversa recupera la primera letra

2.7 Sustitución simple de HOLA con la palabra MURCIELAGO

Enunciado

Cifrar HOLA por sustitución simple con la palabra clave MURCIELAGO.

Datos

Mensaje	HOLA
Clave (palabra)	MURCIELAGO
Modo	Cifrar

Idea del método

En la sustitución simple cada letra del alfabeto se reemplaza siempre por la misma letra de un alfabeto desordenado. Con una palabra clave, el alfabeto cifrado empieza con sus letras (sin repetir) y sigue con las que faltan, en orden. Hay 27! alfabetos posibles, pero el cifrado se rompe por análisis de frecuencias porque cada letra conserva su frecuencia.

alfabeto cifrado = letras de la clave sin repetir + letras restantes en orden

Procedimiento

- 1. Qué hacemos.** Cada letra se cambia por la que ocupa su misma posición en un alfabeto desordenado, que se construye a partir de la clave.
- 2. Construimos el alfabeto cifrado.** Empieza con las letras de la clave («MURCIELAGO») y sigue con las 17 letras que faltan, en orden alfabético: «MURCIELAGOBDFHJKNÑPQSTVWXYZ».
- 3. Tabla de correspondencias.** Arriba va el alfabeto normal (A, B, C...) y abajo el cifrado, letra bajo letra. Cifrar es bajar de fila; descifrar, subir.
- 4. Cambiamos cada letra.** Buscamos cada letra en la fila de arriba y la cambiamos por la de abajo: H -> A, O -> K, L -> D, A -> M.
- 5. Resultado.** «HOLA» se convierte en «AKDM».
- 6. Comprobación.** Leyendo la tabla en sentido contrario, A -> H, y así se recupera «HOLA».

Tabla

A	B	C	D	E	F	G	H	I	J	K	L	M	N	Ñ	O	P	Q	R	S	T	U	V	W	X	Y	Z
M	U	R	C	I	E	L	A	G	O	B	D	F	H	J	K	N	Ñ	P	Q	S	T	V	W	X	Y	Z

Tabla de correspondencias en dos filas: Arriba el alfabeto original y abajo el cifrado. Cifrar baja de fila; descifrar sube.

Resultado

AKDM

Comprobación: A -> H leyendo la tabla al revés: La sustitución inversa recupera la primera letra

3 Criptografía moderna

La criptografía moderna se apoya en problemas matemáticos difíciles: El logaritmo discreto (Diffie-Hellman) y la factorización de números grandes (RSA). La exponenciación rápida es la herramienta que hace posibles esas potencias enormes módulo n con pocas multiplicaciones.

3.1 Diffie-Hellman con $p = 23$, $g = 5$, $a = 6$ y $b = 15$

Enunciado

Con $p = 23$ y $g = 5$ públicos, y las claves privadas $a = 6$ (Alicia) y $b = 15$ (Bob), calcular las claves públicas A y B y la clave compartida K .

Datos

Primo p (público)	23
Generador g (público)	5
Clave privada de Alicia, a	6
Clave privada de Bob, b	15

Idea del método

Diffie-Hellman permite que dos personas acuerden un secreto por un canal público. Cada una publica una potencia de g con su exponente secreto y eleva la potencia recibida al suyo; las dos llegan al mismo K porque $(g^b)^a = (g^a)^b$. Quien escucha ve p , g , A y B , pero recuperar a o b es el problema del logaritmo discreto, intratable con primos grandes. Todas las potencias se calculan con la exponenciación rápida (3.3).

$$\begin{aligned} A &= g^a \bmod p \\ B &= g^b \bmod p \\ K &= B^a \bmod p = A^b \bmod p \end{aligned}$$

Procedimiento

- 1. Qué buscamos.** Que Alicia y Bob acuerden una clave secreta K hablando por un canal que cualquiera puede escuchar. Cada uno guarda un número privado y publica solo una potencia de g .
- 2. Datos.** Públicos: $p = 23$ (el módulo) y $g = 5$ (la base). Privados: $a = 6$ de Alicia y $b = 15$ de Bob.
- 3. Comprobamos p .** Probamos dividir 23 entre 2, 3, 5, ... hasta su raíz cuadrada y ninguno lo divide exacto: 23 es primo, como pide el método.
- 4. Alicia publica A .** Eleva g a su número privado: $A = g^a \bmod p = 5^6 \bmod 23 = 8$. Lo calcula con la exponenciación rápida del 3.3 (4 multiplicaciones).
- 5. Bob publica B .** Hace lo mismo con el suyo: $B = g^b \bmod p = 5^{15} \bmod 23 = 19$ (7 multiplicaciones).

6. **Intercambio.** A y B viajan por el canal público y cualquiera puede verlos; a y b nunca salen de cada lado.
7. **Alicia calcula K.** Eleva lo que recibió de Bob a su número privado: $K = B^a \text{ mod } p = 19^6 \text{ mod } 23 = 2$.
8. **Bob calcula K.** Eleva lo que recibió de Alicia al suyo: $K = A^b \text{ mod } p = 8^{15} \text{ mod } 23 = 2$.
9. **Comprobación.** Los dos obtienen $K = 2$ porque $B^a = (g^b)^a = g^{(a \times b)} = (g^a)^b = A^b \text{ (mod } p)$: Da igual el orden en que se aplican los exponentes. Quien escucha el canal solo ve p, g, A y B; sacar a o b de ahí es el problema del logaritmo discreto, muy difícil con números grandes.

Tabla

Quién	Calcula	Operación	Resultado	Viaja por el canal
Alicia	$A = g^a \text{ mod } p$	$5^6 \text{ mod } 23$	8	sí: A es pública
Bob	$B = g^b \text{ mod } p$	$5^{15} \text{ mod } 23$	19	sí: B es pública
Alicia	$K = B^a \text{ mod } p$	$19^6 \text{ mod } 23$	2	no: K queda en secreto
Bob	$K = A^b \text{ mod } p$	$8^{15} \text{ mod } 23$	2	no: K queda en secreto

A y B viajan por el canal público; a, b y K nunca. Los dos caminos llegan a la misma K porque $(g^b)^a = (g^a)^b = g^{(a \times b)} \text{ (mod } p)$.

Exponenciación rápida [3.3] para $K = B^a \text{ mod } p = 19^6 \text{ mod } 23$

Bit i	Bit del exponente	Potencia al cuadrado	Acumulado
0	0	19	1
1	1	16	16
2	1	3	2

La misma tabla del submenú 3.3: La potencia se eleva al cuadrado en cada fila y el acumulado multiplica las potencias de los bits en 1 del exponente $a = 6$.

Resultado

2

Comprobación: Los dos caminos coinciden: $B^a = 19^6 = 2$ y $A^b = 8^{15} = 2 \text{ (mod } 23)$

3.2 RSA con $p = 61$, $q = 53$, $e = 17$ y $m = 65$

Enunciado

Generar las claves RSA con $p = 61$, $q = 53$ y $e = 17$, cifrar el mensaje $m = 65$ y comprobar que la clave privada lo recupera.

Datos

Primo p	61
Primo q	53
Exponente e	17
Mensaje m	65

Idea del método

RSA usa dos primos secretos p y q . Su producto n es público, pero $\phi(n) = (p - 1)(q - 1)$ solo se conoce si se conocen p y q , y de $\phi(n)$ sale la clave privada $d = e^{-1} \bmod \phi(n)$, con la tabla del AEE (1.6). Cifrar es elevar a a e módulo n y descifrar es elevar a a d ; funciona porque $e \times d = 1 \pmod{\phi(n)}$. La seguridad descansa en que factorizar n es muy difícil cuando los primos son grandes.

$$\begin{aligned} n &= p \times q \\ \phi(n) &= (p - 1)(q - 1) \\ e \times d &= 1 \pmod{\phi(n)} \\ c &= m^e \bmod n \\ m &= c^d \bmod n \end{aligned}$$

Procedimiento

- 1. Qué buscamos.** Crear las claves RSA a partir de los primos p y q , cifrar el mensaje $m = 65$ con la clave pública y comprobar que la clave privada lo recupera.
- 2. Comprobamos los primos.** Probamos dividir 61 y 53 entre 2, 3, 5, ... hasta su raíz cuadrada y ninguno los divide exacto: Los dos son primos, y son distintos.
- 3. Calculamos n .** $n = p \times q = 61 \times 53 = 3233$. Es el módulo de todas las cuentas y forma parte de la clave pública.
- 4. Calculamos $\phi(n)$.** $\phi(n) = (p - 1) \times (q - 1) = 60 \times 52 = 3120$. Es el número que permite hallar la clave privada, y solo se conoce si se conocen p y q .
- 5. Validamos e .** $e = 17$ cumple $1 < 17 < 3120$ y, con el Euclides del 1.4, $\text{MCD}(17, 3120) = 1$: e no comparte divisores con $\phi(n)$. La clave pública es $(n, e) = (3233, 17)$.
- 6. Hallamos d .** d es el inverso de e módulo $\phi(n)$. La tabla del Euclides extendido del 1.6 (4 rondas) da $t = -367$, y $d = -367 \bmod 3120 = 2753$. La clave privada es $d = 2753$.

7. **Comprobamos d.** $17 \times 2753 = 46801 = 15 \times 3120 + 1$: El residuo es 1, como debe ser.
8. **Ciframos.** $c = m^e \bmod n = 65^{17} \bmod 3233 = 2790$, calculado con la exponenciación rápida del 3.3 (6 multiplicaciones).
9. **Comprobación.** Desciframos con la clave privada: $c^d \bmod n = 2790^{2753} \bmod 3233 = 65$, y recuperamos $m = 65$.

Tabla

Bit i	Bit del exponente	Potencia al cuadrado	Acumulado
0	1	65	65
1	0	992	65
2	0	1232	65
3	0	1547	65
4	1	789	2790

La tabla de la exponenciación rápida (3.3) para el cifrado $c = 65^{17} \bmod 3233$: El exponente $e = 17$ en binario es 10001 y solo se multiplican las potencias de sus bits en 1.

Tabla del AEE (1.6) para $d = e^{-1} \bmod \phi(n) = 2753$

Ronda	Cociente q	Residuo r	s	t
Inicio	—	3120	1	0
Inicio	—	17	0	1
1	183	9	1	-183
2	1	8	-1	184
3	1	1	2	-367
4	8	0	-17	3120

La misma tabla del submenú 1.6 con $n = \phi(n) = 3120$ y $a = e = 17$: El inverso es el t de la fila con residuo r = 1, $t = -367$, y $-367 \bmod 3120 = 2753$.

Resultado

2790

Comprobación: $2790^{2753} \bmod 3233 = 65$: El descifrado recupera el mensaje $m = 65$

3.3 $3^{13} \bmod 7$ con cuadrados sucesivos

Enunciado

Calcular $3^{13} \bmod 7$ con el método de cuadrados sucesivos e indicar cuántas multiplicaciones se hacen.

Datos

Base b	3
Exponente e	13
Módulo n	7

Idea del método

La exponenciación rápida escribe el exponente en binario y eleva la base al cuadrado una y otra vez (b , b^2 , b^4 , b^8 ...), reduciendo módulo n en cada paso para que los números no crezcan. Solo se multiplican al acumulado las potencias cuyo bit vale 1. Así se hacen unas pocas multiplicaciones en vez de $e - 1$; es la pieza que usan Diffie-Hellman y RSA.

$b^e \bmod n$: e en binario, cuadrados sucesivos y producto de los bits en 1

Procedimiento

- 1. Qué buscamos.** $3^{13} \bmod 7$: El residuo de esa potencia al dividirla entre 7, sin multiplicar 3 por sí mismo 12 veces.
- 2. El exponente en binario.** $13 = 1101$ (bin), con 4 bits. Lo leemos de derecha a izquierda ($i = 0, 1, 2, \dots$): Cada bit dice si hay que usar la potencia de esa fila.
- 3. Cómo funciona.** Empezamos con la potencia 3. En cada fila la elevamos al cuadrado y tomamos mod 7: Así conseguimos 3^1 , 3^2 , 3^4 , 3^8 ... Cuando el bit vale 1, multiplicamos esa potencia al acumulado (que empieza en 1); cuando vale 0, el acumulado no cambia.
- 4. Bit $i = 0$, vale 1.** La potencia empieza en 3. El bit es 1: $\text{acumulado} = 1 \times 3 = 3$, y $3 \bmod 7 = 3$.
- 5. Bit $i = 1$, vale 0.** Potencia = $3^2 = 9$, y $9 \bmod 7 = 2$. El bit es 0: El acumulado sigue en 3.
- 6. Bit $i = 2$, vale 1.** Potencia = $2^2 = 4$, y $4 \bmod 7 = 4$. El bit es 1: $\text{acumulado} = 3 \times 4 = 12$, y $12 \bmod 7 = 5$.
- 7. Bit $i = 3$, vale 1.** Potencia = $4^2 = 16$, y $16 \bmod 7 = 2$. El bit es 1: $\text{acumulado} = 5 \times 2 = 10$, y $10 \bmod 7 = 3$.
- 8. Resultado.** El acumulado final es la respuesta: $3^{13} \bmod 7 = 3$.
- 9. Cuántas cuentas.** 6 multiplicaciones (3 al cuadrado y 3 por bits en 1); el método directo haría 12.

Tabla

Bit i	Bit del exponente	Potencia al cuadrado	Acumulado
0	1	3	3
1	0	2	3

Bit i	Bit del exponente	Potencia al cuadrado	Acumulado
2	1	4	5
3	1	2	3

El exponente en binario es 1101: La tabla lo recorre al revés, del bit $i = 0$ (el de la derecha) al de la izquierda. La potencia se eleva al cuadrado en cada fila y el acumulado solo cambia cuando el bit es 1.

Resultado

3

Comprobación: Método directo: $3^{13} = 1594323$ y $1594323 \bmod 7 = 3$

4 Algoritmos hash

Una función hash resume cualquier texto en una huella de tamaño fijo: Siempre la misma para el mismo texto, imposible de invertir y muy sensible a cualquier cambio (efecto avalancha). MD5, SHA-256 y SHA-512 se diferencian en el tamaño de la huella y en su seguridad.

4.1 MD5 de «abc» y efecto avalancha frente a «abd»

Enunciado

Calcular el MD5 del texto «abc» y compararlo con el de «abd» para observar el efecto avalancha.

Datos

Texto	abc
Algoritmo	MD5
Texto para comparar	abd

Idea del método

Una función hash convierte cualquier texto en una huella de tamaño fijo. Tres propiedades la definen: El mismo texto da siempre la misma huella, no se puede volver de la huella al texto y cualquier cambio en el texto cambia la huella por completo (efecto avalancha). MD5 produce 128 bits (32 dígitos hexadecimales); hoy se considera roto para seguridad, pero es ideal para estudiar cómo funciona un hash.

MD5(texto) -> 128 bits = 32 dígitos hexadecimales

Procedimiento

- 1. Qué hacemos.** Calcular la huella MD5 del texto: Un resumen de tamaño fijo que cambia por completo si el texto cambia en algo, y desde el que no se puede volver al texto.
- 2. El texto en bytes.** En UTF-8, «abc» ocupa 3 bytes (en hexadecimal): «a» = 61, «b» = 62, «c» = 63.
- 3. El algoritmo mezcla los bytes.** MD5 completa los bytes hasta un bloque de 512 bits y los pasa por muchas rondas de sumas, rotaciones y operaciones lógicas; el estado final de esa mezcla es la huella.
- 4. La huella.** Mide 128 bits y se escribe con 32 dígitos hexadecimales (cada dígito son 4 bits): MD5(«abc») = 900150983cd24fb0d6963f7d28e17f72.
- 5. Segundo texto.** Calculamos también MD5(«abd») = 4911e516e5aa21d327512e0c8b197616.
- 6. Efecto avalancha.** Comparamos los dos hashes posición por posición: Cambian 30 de 32 caracteres (94 %). Un cambio mínimo en el texto cambia casi toda la huella.
- 7. Comprobación.** Calcular MD5 de «abc» otra vez da exactamente la misma huella: El hash no tiene nada de azar.

Detalle

Texto 1: 900150983cd24fb0d6963f7d28e17f72
 Texto 2: 4911e516e5aa21d327512e0c8b197616
 ^^^ ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ ^^^

Resultado

900150983cd24fb0d6963f7d28e17f72

Comprobación: Efecto avalancha: Con dos textos distintos cambian 30 de 32 caracteres del hash

4.2 SHA-256 de «abc»

Enunciado

Calcular el SHA-256 del texto «abc».

Datos

Texto	abc
Algoritmo	SHA-256

Idea del método

SHA-256 pertenece a la familia SHA-2 y produce huellas de 256 bits (64 dígitos hexadecimales). Es el hash más usado hoy: Aparece en los certificados TLS, en las firmas digitales y en la cadena de bloques de Bitcoin. Sus propiedades son las de toda función hash: Determinista, irreversible y con efecto avalancha.

SHA-256(texto) -> 256 bits = 64 dígitos hexadecimales

Procedimiento

- 1. Qué hacemos.** Calcular la huella SHA-256 del texto: Un resumen de tamaño fijo que cambia por completo si el texto cambia en algo, y desde el que no se puede volver al texto.
- 2. El texto en bytes.** En UTF-8, «abc» ocupa 3 bytes (en hexadecimal): «a» = 61, «b» = 62, «c» = 63.
- 3. El algoritmo mezcla los bytes.** SHA-256 completa los bytes hasta un bloque de 512 bits y los pasa por muchas rondas de sumas, rotaciones y operaciones lógicas; el estado final de esa mezcla es la huella.
- 4. La huella.** Mide 256 bits y se escribe con 64 dígitos hexadecimales (cada dígito son 4 bits):
 SHA-256(«abc») = ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad.
- 5. Comprobación.** Calcular SHA-256 de «abc» otra vez da exactamente la misma huella: El hash no tiene nada de azar.

Resultado

ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad

Comprobación: Mismo texto, mismo hash: SHA-256 de «abc» siempre da esta huella

4.3 SHA-512 de «abc»

Enunciado

Calcular el SHA-512 del texto «abc».

Datos

Texto	abc
Algoritmo	SHA-512

Idea del método

SHA-512 es la variante grande de SHA-2: Trabaja con palabras de 64 bits, mezcla los bytes en bloques de 1024 bits y produce huellas de 512 bits (128 dígitos hexadecimales). Ofrece más margen de seguridad que SHA-256 y en procesadores de 64 bits suele ser igual de rápido.

SHA-512(texto) -> 512 bits = 128 dígitos hexadecimales

Procedimiento

- 1. Qué hacemos.** Calcular la huella SHA-512 del texto: Un resumen de tamaño fijo que cambia por completo si el texto cambia en algo, y desde el que no se puede volver al texto.
- 2. El texto en bytes.** En UTF-8, «abc» ocupa 3 bytes (en hexadecimal): «a» = 61, «b» = 62, «c» = 63.
- 3. El algoritmo mezcla los bytes.** SHA-512 completa los bytes hasta un bloque de 1024 bits y los pasa por muchas rondas de sumas, rotaciones y operaciones lógicas; el estado final de esa mezcla es la huella.
- 4. La huella.** Mide 512 bits y se escribe con 128 dígitos hexadecimales (cada dígito son 4 bits):
SHA-512[«abc»] =
ddaf35a193617abacc417349ae20413112e6fa4e89a97ea20a9eeee64b55d39a2192992a274fc1a836ba3c23a3feebd454d4423643ce80e2a9ac94fa54ca49f.
- 5. Comprobación.** Calcular SHA-512 de «abc» otra vez da exactamente la misma huella: El hash no tiene nada de azar.

Resultado

ddaf35a193617abacc417349ae20413112e6fa4e89a97ea20a9eeee64b55d39a2192992a274fc
1a836ba3c23a3feebbd454d4423643ce80e2a9ac94fa54ca49f

Comprobación: Mismo texto, mismo hash: SHA-512 de «abc» siempre da esta huella

5 Codificación

Codificar no es cifrar: No hay clave ni secreto, solo una forma distinta de escribir los mismos bytes. ASCII, hexadecimal, binario y Base64 son representaciones que cualquiera puede deshacer y aparecen por todas partes en informática y en seguridad.

5.1 «Hola» en códigos ASCII decimales

Enunciado

Codificar el texto «Hola» como códigos ASCII decimales separados por espacio.

Datos

Texto	Hola
Modo	Codificar

Idea del método

ASCII asigna a cada carácter un número del 0 al 127: Las mayúsculas van de 65 a 90, las minúsculas de 97 a 122 y el espacio es 32. Codificar es escribir ese número por cada carácter. Los caracteres que no están en ASCII, como la ñ o las tildes, se representan con sus bytes UTF-8 (dos por carácter).

carácter -> código decimal de 0 a 127

Procedimiento

- 1. Qué hacemos.** Cada carácter tiene un número en la tabla ASCII (del 0 al 127): La A es 65, la a es 97, el espacio es 32. Escribimos ese número en decimal.
- 2. Buscamos cada carácter en la tabla.** «H» = 72, «o» = 111, «l» = 108, «a» = 97.
- 3. Resultado.** Los códigos separados por un espacio: 72 111 108 97.
- 4. Comprobación.** Cada código vuelve a su carácter (72 -> «H» ...) y se recupera «Hola».

Tabla

Carácter	Decimal	Hexadecimal	Binario
H	72	48	01001000
o	111	6F	01101111
l	108	6C	01101100
a	97	61	01100001

La columna Decimal es el código de cada byte. Hasta 127 es ASCII puro; de 128 en adelante son bytes UTF-8 de caracteres como la ñ.

Resultado

72 111 108 97

Comprobación: «H» <-> 72: La ida y la vuelta coinciden

5.2 «Hola» en hexadecimal

Enunciado

Codificar el texto «Hola» en hexadecimal.

Datos

Texto	Hola
Modo	Codificar

Idea del método

Hexadecimal es la base 16: Usa los dígitos 0 a 9 y las letras A a F (10 a 15). Un byte vale de 0 a 255 y, como $16 \times 16 = 256$, cada byte cabe exactamente en dos dígitos hexadecimales: El primero es el cociente de dividir entre 16 y el segundo, el residuo. Es la forma habitual de escribir bytes, hashes y direcciones de memoria.

$\text{byte} = d1 \times 16 + d0 \rightarrow \text{dos dígitos hexadecimales } d1 \ d0$

Procedimiento

- 1. Qué hacemos.** Hexadecimal es contar en base 16, con los dígitos 0 a 9 y las letras A a F (A vale 10, F vale 15). Cada byte del texto, un número de 0 a 255, se escribe con exactamente dos dígitos hexadecimales, de 00 a FF.
- 2. El texto en bytes.** En UTF-8, «Hola» ocupa 4 bytes; en decimal: 72 111 108 97.
- 3. Cada byte a dos dígitos.** Por ejemplo «H»: $72 = 4 \times 16 + 8$, y esos dos números escritos como dígitos hexadecimales son 4 y 8: 48. Todos: «H» = 48, «o» = 6F, «l» = 6C, «a» = 61.
- 4. Resultado.** Juntamos las parejas de dígitos: 486F6C61.
- 5. Comprobación.** Leyendo las parejas al revés (48 -> 72 -> «H» ...) se recupera «Hola».

Tabla

Carácter	Decimal	Hexadecimal	Binario
H	72	48	01001000
o	111	6F	01101111

Carácter	Decimal	Hexadecimal	Binario
l	108	6C	01101100
a	97	61	01100001

Cada fila es un byte del texto en UTF-8: El mismo valor escrito en decimal, en hexadecimal y en binario.

Resultado

486F6C61

Comprobación: «H» <-> 48: La ida y la vuelta coinciden

5.3 «Hola» en binario de 8 bits por byte

Enunciado

Codificar el texto «Hola» en binario, con 8 bits por byte.

Datos

Texto	Hola
Modo	Codificar

Idea del método

El computador guarda cada byte como 8 bits, y cada bit es un 0 o un 1. Los bits valen 128, 64, 32, 16, 8, 4, 2 y 1 de izquierda a derecha: El byte es la suma de los valores de los bits en 1. Codificar en binario es escribir esos 8 bits por cada byte del texto en UTF-8.

$$\text{byte} = b7 \times 128 + b6 \times 64 + b5 \times 32 + b4 \times 16 + b3 \times 8 + b2 \times 4 + b1 \times 2 + b0 \times 1$$

Procedimiento

- 1. Qué hacemos.** El computador guarda cada carácter como un byte: 8 bits, y cada bit es un 0 o un 1. Escribimos los 8 bits de cada byte del texto.
- 2. El texto en bytes.** En UTF-8, «Hola» ocupa 4 bytes; en decimal: 72 111 108 97.
- 3. Cada byte a 8 bits.** Los bits valen 128, 64, 32, 16, 8, 4, 2 y 1, de izquierda a derecha, y se pone un 1 en cada valor que hace falta sumar. Por ejemplo 72 = 64 + 8, que se escribe 01001000. Todos: «H» = 01001000, «o» = 01101111, «l» = 01101100, «a» = 01100001.
- 4. Resultado.** Los bytes separados por un espacio para poder leerlos: 01001000 01101111 01101100 01100001.

5. **Comprobación.** Sumando los valores de los bits en 1 de cada grupo se recuperan los bytes (01001000 -> 72 -> «H» ...) y el texto «Hola».

Tabla

Carácter	Decimal	Hexadecimal	Binario
H	72	48	01001000
o	111	6F	01101111
l	108	6C	01101100
a	97	61	01100001

Cada fila es un byte del texto en UTF-8: El mismo valor escrito en decimal, en hexadecimal y en binario.

Resultado

01001000 01101111 01101100 01100001

Comprobación: «H» <-> 01001000: La ida y la vuelta coinciden

5.4 «Hola» en Base64

Enunciado

Codificar el texto «Hola» en Base64.

Datos

Texto	Hola
Modo	Codificar

Idea del método

Base64 sirve para escribir bytes cualesquiera usando solo 64 caracteres seguros (A-Z, a-z, 0-9, + y /), por ejemplo para adjuntar archivos en un correo o incrustar imágenes en HTML. Toma los bits de 6 en 6 ($2^6 = 64$) y cada grupo elige un carácter; el resultado va en bloques de 4 caracteres y el relleno «=» completa el último. Ocupa un tercio más que los bytes originales.

3 bytes = 24 bits -> 4 grupos de 6 bits -> 4 caracteres

Procedimiento

1. **Qué hacemos.** Base64 escribe cualquier secuencia de bytes usando solo 64 caracteres (A-Z, a-z, 0-9, + y /). Toma los bits de 6 en 6 y cada grupo de 6 bits, leído como un número de 0 a 63, elige un carácter.

2. **El texto en bits.** En UTF-8, «Hola» ocupa 4 bytes, es decir 32 bits: 01001000 01101111 01101100 01100001.
3. **Partimos en grupos de 6.** Escribimos los bits seguidos y los cortamos de 6 en 6: 010010 000110 111101 101100 011000 010000 (6 grupos; los últimos 4 bits se completan con ceros).
4. **Cada grupo elige un carácter.** 010010 = 18 -> «S», 000110 = 6 -> «G», 111101 = 61 -> «9», 101100 = 44 -> «s», 011000 = 24 -> «Y», 010000 = 16 -> «Q».
5. **Relleno.** Los caracteres van en bloques de 4; faltan 2 y se completan con «=».
6. **Resultado.** SG9sYQ==.
7. **Comprobación.** Decodificar SG9sYQ== devuelve «Hola».

Tabla

Grupo de 6 bits	Decimal	Carácter Base64
010010	18	S
000110	6	G
111101	61	9
101100	44	s
011000	24	Y
010000	16	Q
(relleno)	—	=
(relleno)	—	=

Cada carácter Base64 guarda 6 bits. Las filas de relleno «=» no llevan bits: Solo completan el bloque de 4 caracteres.

Los bytes del texto, bit a bit

Carácter	Decimal	Hexadecimal	Binario
H	72	48	01001000
o	111	6F	01101111
l	108	6C	01101100
a	97	61	01100001

De aquí salen los bits que arriba se agrupan de 6 en 6.

Resultado

SG9sYQ==

Comprobación: 010010 = 18 = «S»; Cada carácter Base64 guarda exactamente 6 bits

6 Uso de salt

Las claves de los usuarios no se guardan en claro ni con un hash simple: Se les añade un salt aleatorio y único antes de calcular el hash. Así dos usuarios con la misma clave tienen hashes distintos y las tablas precalculadas de hashes dejan de servir.

6.1 Verificar «Clave123» con el salt a1b2c3d4 y MD5

Enunciado

Comprobar si la clave «Clave123» es correcta a partir del salt a1b2c3d4 y el hash MD5 guardado.

Datos

Clave a comprobar	Clave123
Salt guardado	a1b2c3d4
Hash guardado	3baad75a2e328110d25f895f12abc632
Algoritmo	MD5

Idea del método

Un sistema no debe guardar las claves de sus usuarios: Guarda un salt (un valor aleatorio único por usuario) y el hash de salt + clave. Para verificar una clave repite la fórmula con el salt guardado y compara el resultado con el hash guardado. El salt hace que dos usuarios con la misma clave tengan hashes distintos y que las tablas de hashes precalculados no sirvan. En la práctica se usan funciones lentas como bcrypt, Argon2 o PBKDF2.

```
hash = MD5(salt + clave)
```

Procedimiento

1. **Qué hacemos.** Comprobar la clave sin tenerla guardada: Repetimos la fórmula $\text{hash} = \text{MD5}(\text{salt} + \text{clave})$ con el salt guardado y comparamos el resultado con el hash guardado.
2. **Pegamos salt y clave.** «a1b2c3d4» + «Clave123» = «a1b2c3d4Clave123».
3. **Calculamos el hash.** $\text{MD5}(\text{«a1b2c3d4Clave123»}) = 3baad75a2e328110d25f895f12abc632$.
4. **Comparamos.** Coincide con el hash guardado carácter a carácter: La clave es correcta.

Detalle

Guardado: 3baad75a2e328110d25f895f12abc632
 Calculado: 3baad75a2e328110d25f895f12abc632

Resultado

3baad75a2e328110d25f895f12abc632

Comprobación: MD5(salt + clave) reproduce el hash guardado: La clave es correcta

6.2 Verificar «Clave123» con el salt a1b2c3d4 y SHA-256

Enunciado

Comprobar si la clave «Clave123» es correcta a partir del salt a1b2c3d4 y el hash SHA-256 guardado.

Datos

Clave a comprobar	Clave123
Salt guardado	a1b2c3d4
Hash guardado	e3143dc97e4a38f053866bbd28780a1a90af33199dba6cb08681dced52685236
Algoritmo	SHA-256

Idea del método

Igual que con MD5, el sistema guarda el salt y el hash de salt + clave, nunca la clave. Con SHA-256 la huella mide 256 bits (64 dígitos hexadecimales), lo que hace inviable adivinarla por fuerza bruta. El salt, único por usuario, evita que dos claves iguales produzcan el mismo hash y anula las tablas precalculadas.

```
hash = SHA-256(salt + clave)
```

Procedimiento

- 1. Qué hacemos.** Comprobar la clave sin tenerla guardada: Repetimos la fórmula $\text{hash} = \text{SHA-256}(\text{salt} + \text{clave})$ con el salt guardado y comparamos el resultado con el hash guardado.
- 2. Pegamos salt y clave.** «a1b2c3d4» + «Clave123» = «a1b2c3d4Clave123».
- 3. Calculamos el hash.** $\text{SHA-256}(\text{«a1b2c3d4Clave123»}) = \text{e3143dc97e4a38f053866bbd28780a1a90af33199dba6cb08681dced52685236}$.
- 4. Comparamos.** Coincide con el hash guardado carácter a carácter: La clave es correcta.

Detalle

Guardado: e3143dc97e4a38f053866bbd28780a1a90af33199dba6cb08681dced52685236
 Calculado: e3143dc97e4a38f053866bbd28780a1a90af33199dba6cb08681dced52685236

Resultado

e3143dc97e4a38f053866bbd28780a1a90af33199dba6cb08681dced52685236

Comprobación: SHA-256(salt + clave) reproduce el hash guardado: La clave es correcta

6.3 Verificar «Clave123» con el salt a1b2c3d4 y SHA-512

Enunciado

Comprobar si la clave «Clave123» es correcta a partir del salt a1b2c3d4 y el hash SHA-512 guardado.

Datos

Clave a comprobar	Clave123
Salt guardado	a1b2c3d4
Hash guardado	42f05bf6608218bfbe7339547035e09ce95ace42b1e850d8e967a2da b75954fa84da5f396a7412dda064d8ffcb815c5e0678cbe772a1a83a 28cf3838d84b7ea8
Algoritmo	SHA-512

Idea del método

Con SHA-512 la huella mide 512 bits (128 dígitos hexadecimales). El procedimiento es el mismo: Pegar el salt delante de la clave, calcular el hash y comparar con el guardado. Si coinciden, la clave es correcta; si no, la clave o el salt no son los originales.

```
hash = SHA-512(salt + clave)
```

Procedimiento

- 1. Qué hacemos.** Comprobar la clave sin tenerla guardada: Repetimos la fórmula $\text{hash} = \text{SHA-512}(\text{salt} + \text{clave})$ con el salt guardado y comparamos el resultado con el hash guardado.
- 2. Pegamos salt y clave.** «a1b2c3d4» + «Clave123» = «a1b2c3d4Clave123».
- 3. Calculamos el hash.** $\text{SHA-512}(\text{«a1b2c3d4Clave123»}) =$
42f05bf6608218bfbe7339547035e09ce95ace42b1e850d8e967a2dab75954fa84da5f396a7412dda064d8ff
cb815c5e0678cbe772a1a83a28cf3838d84b7ea8.
- 4. Comparamos.** Coincide con el hash guardado carácter a carácter: La clave es correcta.

Detalle

Guardado: 42f05bf6608218bfbe7339547035e09ce95ace42b1e850d8e967a2dab75954fa84da5f396a7412dda064d8ffcb815c5e0678cbe772a1a83a28cf3838d84b7ea8
 Calculado: 42f05bf6608218bfbe7339547035e09ce95ace42b1e850d8e967a2dab75954fa84da5f396a7412dda064d8ffcb815c5e0678cbe772a1a83a28cf3838d84b7ea8

Resultado

42f05bf6608218bfbe7339547035e09ce95ace42b1e850d8e967a2dab75954fa84da5f396a7412dda064d8ffcb815c5e0678cbe772a1a83a28cf3838d84b7ea8

Comprobación: SHA-512(salt + clave) reproduce el hash guardado: La clave es correcta